

Mirage：张量程序的多层次超级优化器

Mengdi Wu Xinhao Cheng Shengyu Liu[†] Chunan Shi[†] Jianan Ji
Man Kit Ao Praveen Velliengiri[‡] Xupeng Miao[‡] Oded Padon[◊] Zhihao Jia

卡内基梅隆大学 北京大学[†]
宾夕法尼亚州立大学[‡] 普渡大学[‡] 魏茨曼科学研究院[◊]

Abstract

我们介绍 Mirage，这是第一个面向张量程序的多层次超级优化器。Mirage 的核心思想是 μ 图——一种在 GPU 计算层次结构的核 (kernel)、线程块 (thread block) 和线程 (thread) 三个层次上统一表示张量程序的方法。 μ 图使得 Mirage 能够发现将代数变换、调度变换以及生成新的自定义核相结合的新颖优化方案。为了在庞大的搜索空间中高效导航，Mirage 引入了一种基于抽象的剪枝技术，该技术显著缩减了搜索空间，并提供一定的最优性保证。为了确保优化后的 μ 图与输入程序等价，Mirage 引入了一种具有强理论保证的概率等价验证过程。我们的评估表明，即使对于被广泛使用和深度优化的深度神经网络，Mirage 也显著优于现有方法。Mirage 已公开发布于 <https://github.com/mirage-project/mirage>。

1 引言

在 GPU 上实现深度神经网络 (DNN) 的高性能执行，对于现代机器学习应用至关重要。当今的 DNN 框架通常使用张量程序来描述 DNN 计算，张量程序是有向无环图，其节点和边分别表示张量代数运算符（如矩阵乘法）以及运算符之间共享的张量（即 n 维数组）。

为了优化输入的张量程序，现有框架（如 PyTorch [34] 和 TensorFlow [9]）使用手动设计的规则将张量程序映射到专家编写的 GPU 核。这些方法通常需要大量工程工作来设计和实现优化规则，并且可能会错过某些优化机会。为了应对这些挑战，近年来的研究引入了自动化方法，通过搜索涵盖广泛的程序变换空间并根据目标 GPU 上的实际性能来应用变换，从而优化张量程序。这些方法大体可分为两类。

第一类工作，包括 Halide [35]、TVM [13] 和 Ansor [51]，其出发点是 Halide 中引入的算法与调度分离思想¹，在固定算法的前提下优化张量程序的调度。对于给定的算法，这些优化器通过搜索在目标硬件上执行核的可能策略，自动生成高性能核。然而，由于 DNN 的线性代数本质，张量程序可以用数学上等价的多种算法来表示。

¹在调度优化文献中，算法 (algorithm) 描述了核中需要计算什么，而调度 (schedule) 规定了如何执行这个核。

现有的基于调度的优化器只考虑算法由用户手工指定的核，从而错失了优化机会。

第二类工作，包括 TASO、Grappler、Tensat 和 PET，考虑代数变换，即利用张量程序不同算法之间的数学等价性 [3, 25, 46, 48]。代数变换的示例包括：(1) 将一种线性代数运算符转换为另一种，例如将卷积变换为矩阵乘法；(2) 融合多个运算符以减少内存访问和核启动开销；以及 (3) 根据交换律、结合律和分配律重新组织运算符。这些优化器在算法层面执行代数变换，并要求程序员手动指定可用运算符及其实现。因此，它们受限于所提供核的性能。

所有现有的自动化优化方法，无论来自哪一类，都仍然要求程序员手动指定一组核（每个核由一个张量函数定义），然后探索代数变换或调度变换的搜索空间。然而，一些先进的性能优化需要在 GPU 计算层次的核、线程块和线程三个层次上协同进行变换，甚至涉及引入全新的核计算（例如，一个能分解标准核并只融合特定计算的自定义核）。这类优化超出了现有自动化方法的搜索空间，至今仍需手动实现。

其中一个典型例子是 FlashAttention [17]（详见 §8.2），它通过在算法层面重排运算符（代数变换）、重组跨 GPU 核的计算（生成新的自定义核）以及调整每个核的并行化策略以适应 GPU 架构（调度变换），在 GPU 上优化了注意力 [47] 的计算。这个例子所需的变换无法被现有框架自动发现，必须手动实现。FlashAttention 在 Triton [43]（一种广泛使用的张量程序优化器）中的实现，即使使用高级编程语言，也包含超过 700 行代码 [8]。

我们提出 Mirage，这是第一个面向张量程序的多层次超级优化器。Mirage 能够自动发现并验证张量程序的复杂优化方案，这些方案需要联合优化代数变换、调度变换，并发现新的自定义核。

Mirage 的核心思想是 μ 图——一种层次图表示，能够在 GPU 计算层次的多层次上描述张量程序。通过统一处理核、线程块和线程这三个层次， μ 图能够在这些层次上捕获代数变换和调度变换。此外，优化 μ 图还可以引入新的自定义核，这超出了代数变换和调度变换的范畴。例如，Mirage 自动发现了代表 FlashAttention [17] 及其推理变体 FlashDecoding [5] 的 μ 图，以及在某些使用场景中超越这些手动设计的核最高达 $2.2\times$ 的其他

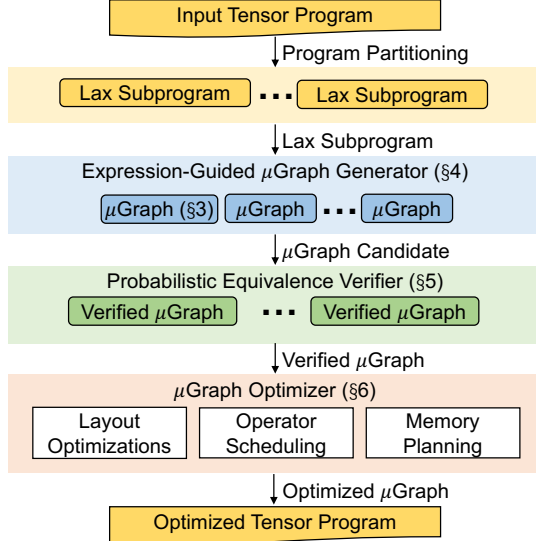


Figure 1: Mirage 系统概览。

μ 图。 Mirage 发现的大多数优化方案超出了现有方法的搜索空间。

Figure 1 展示了 Mirage 的系统概览。Mirage 首先将输入的张量程序拆分为属于受限 LAX 片段的子程序。LAX 片段在 §5 中有正式定义，包括多线性运算符（如矩阵乘法和卷积）、除法（用于归一化）以及有限的指数运算（用于激活函数）。将张量程序划分为 LAX 子程序，能够在保留大多数优化机会的同时缩减每个子程序的优化搜索空间；同时还支持 Mirage 的概率等价验证器。

表达式引导的 μ 图 生成器。 对于每个 LAX 子程序，Mirage 的表达式引导生成器穷举搜索与之等价的所有可能 μ 图。Mirage 必须解决的一个关键挑战是，与先前的超级优化技术相比，其搜索空间要大得多。例如，TASO [25] 和 PET [46] 只在核层次上使用固定的预定义核集合搜索张量程序，而 Mirage 则在核、线程块和线程三个层次上同时考虑超级优化。为了高效导航这个更大的搜索空间，Mirage 引入了一种基于抽象表达式的新型剪枝技术，该技术在提供一定理论最优性保证的同时，大幅减少了 Mirage 需要考察的 μ 图 数量。Mirage 通过将搜索重点放在核和块层次，并对线程层次采用基于规则的方法，进一步缩减了搜索空间。

概率等价验证器。 对于 Mirage 发现的 μ 图，验证其与输入程序的功能等价性带来了另一个挑战，因为程序的输入和输出张量可能包含多达数百万个元素。Mirage 的核心思想是概率等价验证，通过在有限域上执行随机测试来检验 μ 图 之间的等价性。虽然随机测试通常对一般程序只能提供有限的正确性保证，但 Mirage 利用一个新颖的理论结论证明：LAX 片段施加的限制确保了对于 LAX 程序，在有限域上的随机测试能够提供强有力的正确性保证。具体而言，我们证明了多项式恒等测试（PIT）算法 [37, 54] 可以推广到 LAX 程序，从而得到一种可使精度任意高的 LAX 程序等价性随机算法。

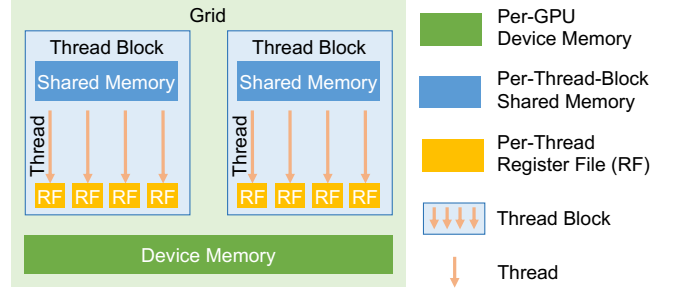


Figure 2: GPU 计算与内存层次结构。

Mirage 使用这一随机算法来（概率性地）确保每个优化后的程序与输入程序等价。

μ 图 优化器。 对于每个经过验证的 μ 图，Mirage 的 μ 图 优化器通过进一步考虑潜在的张量布局、调度运算符执行顺序，以及在核、线程块和线程三个层次上规划内存分配，来最大化其运行时性能。最终，Mirage 根据为每个 LAX 子程序找到的最佳 μ 图，返回优化后的张量程序。

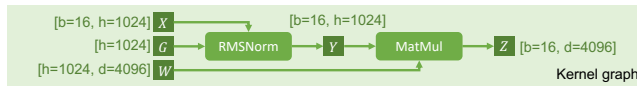
评估结果。 我们在 NVIDIA A100 和 H100 GPU 上针对多种常用 DNN 基准测试评估了 Mirage。即使对于被现有系统广泛使用和深度优化的 DNN 基准（如用于大语言模型（LLM）的分组查询注意力 [41]），Mirage 通过利用现有系统中缺失的微妙自定义核和优化手段，仍能将现有方法的性能提升最高达 3.3×。

2 多层次图表示

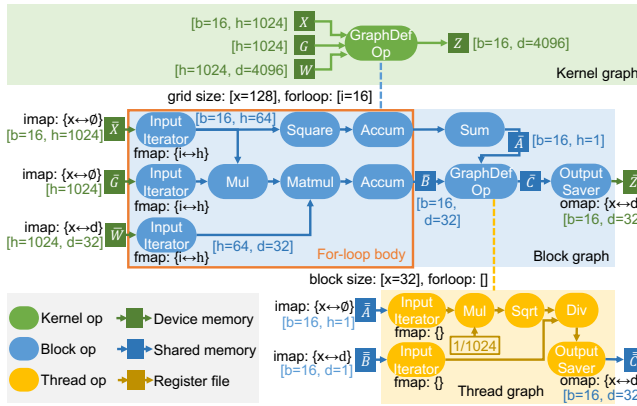
Mirage 使用 μ 图 来描述张量程序在 GPU 上的执行。μ 图 包含多个层次的层次图，以在核（kernel）、块（block）和线程（thread）层次上表示计算²。本节首先介绍 GPU 层次结构，并以 Figure 3 为运行示例，介绍 μ 图 的关键组成部分。

GPU 层次结构。 Figure 2 展示了当今 GPU 的层次结构。GPU 上的计算以核（kernel）的形式组织，每个核是一个以单程序多数据（SPMD）方式在多个 GPU 核心上同时执行的函数。一个核包含一个由线程块（thread block）组成的网格，每个线程块在一个 GPU 流式多处理器上执行，并包含多个线程（thread）来对各个数据元素执行计算。每个线程关联一个线程私有的寄存器文件（register file），同一线程块内的所有线程可以访问共享内存（shared memory）以执行集体操作。最后，核的所有输入和输出都存储在 GPU 的设备内存（device memory）中。

²为简便起见，我们用块（block）指代 CUDA 核的线程块（thread block），用线程（thread）指代单个 CUDA 线程。



(a) RMSNorm 与 MatMul 的计算图。



(b) Mirage 发现的最优 μ 图。

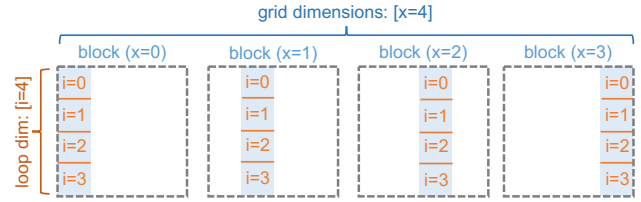
Figure 3: Figure 3a 是 RMSNorm 与 MatMul 的计算图。Figure 3b 展示了 Mirage 为计算 RMSNorm 和 MatMul 所发现的最优 μ 图，该 μ 图将计算融合在单个核中以减少设备内存访问和核启动开销，比现有方法性能提升 1.9 $\times$ 。括号中的数字表示张量形状，花括号中的数字显示对应运算符的 $imap$ 、 $omap$ 或 $fmap$ 。

核图 (Kernel Graph)。 每个张量程序对应一个核图，其中每个节点代表运行在整个 GPU 上的一个核，每条边是核之间共享的张量。核图中的所有张量都存储在 GPU 设备内存中，因为不同的核无法在寄存器文件或共享内存中共享数据。核图中的每个节点可以是预定义的核运算符，由现有核库支持，例如 cuDNN [15] 的卷积和 cuBLAS [16] 的矩阵乘法。此外，为了支持细粒度的核间优化（如核融合），核图中的节点也可以是图定义的核运算符，其语义和行为由一个更低层次（即块）图来定义。例如，Figure 3b 中的核运算符就是一个由块图定义的图定义运算符。

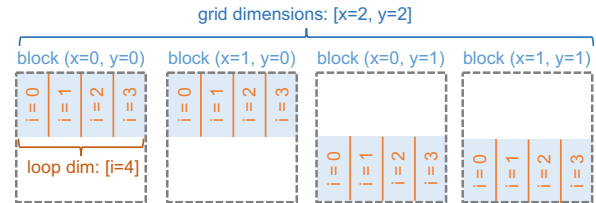
块图 (Block Graph)。 块图描述与一个线程块相关的计算³，其中每个节点表示一个块运算符，指定块内的计算，每条边 (Figure 3b 中的蓝色箭头) 是块运算符之间共享的张量。Mirage 出于以下两方面考虑，将块图中的所有中间张量存储在 GPU 共享内存中。第一，GPU 共享内存比设备内存提供更高的带宽，这一设计使 Mirage 能够通过最大限度地中间结果保存在共享内存中来减少设备内存访问。第二，对于大小超过共享内存容量而必须存储在设备内存中的张量，Mirage 使用这些张量将计算拆分为多个块图，每个块图只包含共享内存中的张量，这种分离不会引入额外的设备内存访问。

每个块图还关联了描述其执行方式的属性，我们将在下面介绍。

³在 CUDA 编程模型中，核的计算被定义为各独立线程块的计算。



(a) $imap = \{x \leftrightarrow \text{column}\}$, $fmap = \{i \leftrightarrow \text{row}\}$



(b) $imap = \{x \leftrightarrow \phi, y \leftrightarrow \text{row}\}$, $fmap = \{i \leftrightarrow \text{column}\}$

Figure 4: 演示如何使用 $imap$ 和 $fmap$ 将输入张量划分到各块和 for 循环迭代中。

网格维度 (Grid Dimensions)。 一个核内的所有块被组织成最多 3 维的网格，由 x 、 y 和 z 维来标识。块图关联最多三个网格维度，指定 x 、 y 和 z 维上的块数。Figure 3b 中的块图启动了 128 个块。

对于图定义核运算符的每个输入张量 (例如 Figure 3b 中核图里的 X 、 G 和 W)，关联的块图包含一个 $imap$ ，用于指定输入张量如何被划分为各个块的子张量。对于每个网格维度 (即 x 、 y 或 z)， $imap$ 将其映射到 (1) 输入张量的某个数据维度，或 (2) 特殊的副本维度 ϕ 。对于情况 (1)，映射的数据维度在该网格维度上的各块之间均等划分；对于情况 (2)，输入张量在这些块之间复制。例如，Figure 3b 中的块图接收三个输入—— $\bar{X}$ 、 $\bar{G}$ 和 $\bar{W}$ ——代表每个块的输入张量。对于 $\bar{W}$ ，其 $imap = \{x \leftrightarrow d\}$ 表示张量 W 的 d 维被划分为 128 个等大小的块。因此， $\bar{W}$ 的形状为 $[h=1024, d=32]$ 。

其次，对于块图的每个输出张量 (如 Figure 3b 中的 $\bar{Z}$)，块图包含一个 $omap$ ，指定如何将所有块的输出拼接以构建核运算符的最终输出。在 $omap$ 中，每个网格维度必须映射到输出张量的某个数据维度，因为不同块必须在设备内存中存储不相交的张量。对于 Figure 3b 中形状为 $[b=16, d=32]$ 的 $\bar{Z}$ ，其 $omap = \{x \leftrightarrow d\}$ 表示具有相同 x 索引的块沿 d 维拼接，从而得到形状为 $[b=16, d=4096]$ 的张量 Z 。

For 循环体。 为了使大型输入张量能够放入共享内存，并将设备内存数据加载与计算重叠，块图可以包含一个 for 循环体，该循环体被反复执行以完成一个核的计算。通常，核中的 for 循环之后会有一些后处理步骤。例如，计算平均值时，for 循环会对 n 个值进行求和，而后处理步骤会除以 n 。Mirage 使用输入迭代器 (input iterators)、for 循环累加器 (for-loop accumulators) 以及二者之间的所有运算符来描述块图的 for 循环体 (如 Figure 3b 中橙色框所示)。块图的每个输入张量首先经过一个输入迭代器，该迭代器将张量的一部分 (如 $\bar{X}$ 、 $\bar{G}$ 和 $\bar{W}$) 从设备内存加载到共享内存中。每个输入迭

代器关联一个 *fmap*，用于指定每次迭代中加载输入张量的哪个部分。形式上，*fmap* 将每个 for 循环维度映射到 (1) 输入张量的某个数据维度，或 (2) 副本维度 ϕ 。与 *imap* 类似，情况 (1) 时沿该维度均等划分，情况 (2) 时复制。Figure 4 展示了如何使用不同的 *imap* 和 *fmap* 将输入矩阵划分到各块和 for 循环迭代中。

每个块图还关联一个 for 循环维度，决定 for 循环体需要执行多少次以完成核的计算。此外，Mirage 使用 for 循环累加器（如 Figure 3b 中的两个 Accum 运算符）来累积每次迭代中计算的中间结果（使用标准累加器，如求和和取最大值），并将累积结果存储在共享内存中。for 循环体执行完毕后，Mirage 继续直接对累积结果执行 for 循环体之外的其余运算符。输出保存器（output saver）随后将共享内存中的最终结果保存回设备内存。

线程图 (Thread Graph)。 线程图进一步将计算范围从块缩小到单个线程。与块图类似，每个线程图也关联了块维度（block dimensions，指定块内线程的组织方式）和 for 循环维度（for-loop dimensions，定义完成指定计算所需的总迭代次数）。每个线程图包含输入迭代器（将输入张量从共享内存加载到寄存器文件中，如 Figure 3b 中的 $\bar{A}$ 和 $\bar{B}$ ）和输出保存器（将输出张量从寄存器文件存储回共享内存，如 $\bar{C}$ ）。线程图是 μ 图中最低层次的图，只包含预定义的线程运算符。

张量布局 (Tensor Layout)。 核图、块图或线程图中的每个张量都关联了一种张量布局（为简便起见，Figure 3 中省略），用于指定张量在内存中的线性化方式。注意，张量布局只影响 μ 图的性能，对输出的正确性没有影响。

定义 2.1 (μ 图合法性)。 若一个 μ 图 G 满足以下条件，则称其为合法的：(1) 对于 G 中每个核、块和线程运算符 o ，其输入和输出张量符合 o 的规范；(2) 每个核、块和线程图中的所有张量分别能够驻留在 GPU 的设备内存、共享内存和寄存器文件中；(3) 对于每个含有 for 循环体的块图和线程图，从输入到输出的每条路径上恰好经过一个输入迭代器、一个 for 循环累加器和一个输出保存器。

与先前工作的比较。 先前工作分别考虑代数变换 [25, 46] 或调度变换 [13, 31, 35]，而 μ 图能够以统一的方式表示二者。具体来说，网格维度和 for 循环维度及其对应的张量维度映射（即 *imap*、*omap* 和 *fmap*）构成了图定义运算符可能调度的全面搜索空间。核、块和线程层次的层次图使得 Mirage 能够在这些层次上探索代数变换。

3 案例研究：RMSNorm

在本节中，我们以均方根层归一化 (RMSNorm) [50] 为案例研究，展示 μ 图表示和 Mirage 超级优化方法的优势。RMSNorm 是近期大型语言模型中广泛使用的归一

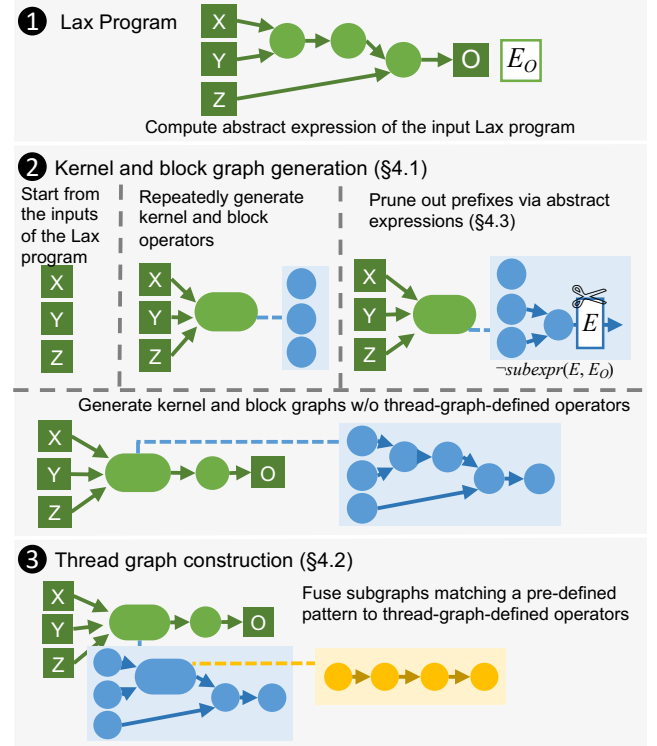


Figure 5: μ 图生成器概览。

化技术 [41]。形式上，RMSNorm 接收两个张量 X 和 G 作为输入，并根据均方根对其逐元素乘积进行归一化：

$$Y_{ij} = \frac{X_{ij}G_j}{\text{RMS}(X_i)}, \quad \text{RMS}(X_i) = \sqrt{\frac{1}{d} \sum_{j=1}^d X_{ij}^2}, \quad (1)$$

其中 d 是 X 的隐藏维度大小。

RMSNorm 之后通常紧跟一个矩阵乘法 (MatMul)。Figure 3a 展示了 RMSNorm 后接 MatMul 运算符的计算图，其中 X 是输入张量， G 和 W 表示两个权重张量。现有机器学习编译器通常为 RMSNorm 和 MatMul 计算分别启动两个独立的核，因为这两个运算在内部都对输入维度进行规约，使得将它们的计算融合到单个核中变得困难。这种方法需要将中间结果（即 Y ）存储在设备内存中，因为不同的核无法在共享内存或寄存器文件中共享数据。

Figure 3b 展示了 Mirage 为将 RMSNorm 和 MatMul 融合在单个核中自动发现的最优 μ 图。该计算被融合在单个图定义核运算符中，以避免将中间结果（即 Y ）保存到设备内存，并减少核启动开销。

我们重点介绍 Mirage 发现的 μ 图与原始 μ 图之间的关键差异。这些差异涉及发现新的自定义核以及代数变换和调度变换的结合，使得通过单独考虑代数变换和调度变换来发现最终 μ 图是不可行的。首先，Mirage 通过利用矩阵乘法和逐元素除法的交换律（代数变换），对 MatMul 和 RMSNorm 的除法进行重排序。其次，Mirage 并行执行均方根中的累加（即 $A_i = \sum_j X_{ij}^2$ ）

和矩阵乘法中的累加（即 $B_{ik} = \sum_j X_{ij} G_j W_{jk}$ ）（调度变换），从而避免将累加结果写入设备内存。接着，Mirage 实例化一个线程图，在寄存器文件中执行一系列逐元素运算符，同时将所有中间结果保持在寄存器文件中（调度变换）。最后，发现的最优 μ 图使用一个新的自定义核来融合 RMSNorm 和 MatMul 的计算，减少设备内存访问和核启动开销。该 μ 图在 NVIDIA A100 和 H100 GPU 上分别比现有系统中的手写核性能提升 $1.5\times$ 和 $1.9\times$ 。

4 表达式引导的 μ 图生成器

Algorithm 1 Mirage 的混合 μ 图生成算法。

输入：具有计算图 G_{ref} 的 LAX 程序
输出：一组 μ 图 S

```

1:  $E_O \leftarrow E(G_{\text{ref}})$ 
2:  $S_0, S \leftarrow \emptyset$ 
3: 生成下一个核运算符 ( $\text{Inputs}(G_{\text{ref}})$ )
4: for all  $G \in S_0$  do
5:    $S \leftarrow S \cup \{\text{线程图构建}(G)\}$ 

6: function 生成下一个核运算符 ( $G_K$ )
7:    $S_0 \leftarrow S_0 \cup \{G_K\}$ 
8:   for all 核图运算符类型  $t$ ; 输入集  $I$  do
9:     if 对  $G_K$  中每个  $op$ ,  $\text{rank}(I, t) > \text{rank}(op.I, op.t)$  then
10:      if  $t$  是预定义运算符 then
11:        if  $o := \text{构建运算符}(G_K, I, t)$  合法 then
12:          生成下一个核运算符 ( $G_K \cup \{o\}$ )
13:      else  $\triangleright t$  是图定义运算符
14:        for all  $\text{gridDims}; \text{forloopDims}$  do
15:           $G_B \leftarrow \text{TBGraph}(I, \text{gridDims}, \text{forloopDims})$ 
16:          生成下一个块运算符 ( $G_K, G_B$ )

17: function 生成下一个块运算符 ( $G_K, G_B$ )
18:   if  $G_B$  中所有共享张量已被消耗 then
19:     if  $o := \text{构建运算符}(G_K, G_B.I, G_B)$  合法 then
20:       生成下一个核运算符 ( $G_K \cup \{o\}$ )
21:   for all 块图运算符类型  $t$ ; 输入集  $I$  do
22:     if 对  $G_B$  中每个  $op$ ,  $\text{rank}(I, t) > \text{rank}(op.I, op.t)$  then
23:       if  $o := \text{构建运算符}(G_B, I, t)$  合法 then
24:         生成下一个块运算符 ( $G_K, G_B \cup \{o\}$ )

25: function 构建运算符 ( $G, I, \text{attrs}$ )
26:    $E \leftarrow \text{表达式推导}(E(I), \text{attrs})$   $\triangleright$  参见表 1
27:   if 子表达式( $E, E_O$ ) then  $\triangleright$  通过抽象表达式剪枝
28:      $S \leftarrow G.\text{outputTensorShapeInfr}(I, \text{attrs})$   $\triangleright$  检查张量形状
29:     if  $S.\text{valid}, G.\text{mAlloc} + S.\text{size} \leq G.\text{mLimit}$  then  $\triangleright$  检查内存
30:       return  $G.\text{constructOp}(I, \text{attrs})$ 
31:   return 非法

32: function 线程图构建 ( $G$ )
33:    $G_{\text{fused}} \leftarrow G$ 
34:   while  $\exists o \in G_{\text{fused}}$  可与其前驱运算符融合 do
35:      $G_{\text{fused}} \leftarrow \text{融合运算符}(G_{\text{fused}}, o)$ 
36:   return  $G_{\text{fused}}$ 

```

本节介绍 Mirage 的 μ 图生成器，它能自动为输入张量程序发现潜在的 μ 图。为了生成能够捕获核、块和线程层次优化的 μ 图，Mirage 必须探索比现有超级优化器大得多的搜索空间，现有超级优化器只考虑核层次

的优化。Mirage 采用两项关键技术来应对这一挑战。第一，基于核和块层次的优化对性能的影响远比线程层次更为关键这一观察——因为访问设备内存和共享内存比访问寄存器文件昂贵几个数量级——Mirage 的 μ 图生成器采用了一种混合方法：在核和块层次穷举考虑所有不超过一定大小的可能图，并使用基于规则的策略来构建线程层次的图。这一方法在保留大多数性能关键优化的同时缩减了搜索空间。第二，为了进一步剪枝搜索空间，Mirage 引入了一种基于 μ 图抽象的剪枝技术，称为抽象表达式，在提供一定理论最优性保证的同时减少了 Mirage 需要考察的 μ 图数量。我们在 §4.1 和 §4.2 中介绍混合 μ 图生成算法，在 §4.3 中介绍表达式引导的剪枝技术。

4.1 核图和块图生成

Mirage 增量式地生成核图和块图，并利用多种剪枝技术来缩减搜索空间，如 Figure 5 第二部分所示。具体来说，Mirage 维护一个合法 μ 图的前缀，并通过迭代方式用新运算符扩展它。对于图 $G = (V, E)$ ，若其子图 $G' = (V', E')$ 满足 $\forall u \in V', \forall (v, u) \in E, v \in V'$ ，则称 G' 为 G 的一个前缀。

为了生成核图中的下一个运算符，Mirage 枚举核运算符类型 t 和输入张量集 I 。若 t 表示图定义运算符类型，Mirage 将通过 (1) 枚举网格维度和 for 循环维度（见 §2），计算块图的输入张量形状；以及 (2) 执行与核层次类似但不考虑图定义运算符的嵌套生成过程，来生成定义其核计算的关联块图。Algorithm 1 中的第 6–16 行和第 17–24 行分别展示了 Mirage 如何生成核运算符和块运算符。

Mirage 在添加运算符之前检查张量形状（第 28 行）和内存使用（第 29 行），以确保前缀合法。

为了确保相同的 μ 图只被生成一次，Mirage 定义了 μ 图的标准形式。对于一个算子按拓扑顺序为 $o_1, \dots, o_n$ 的 μ 图 G ， o_i 的第 j 个输出的索引定义为元组 (i, j) 。 G 中每个运算符 o_i 被赋予一个秩 (rank) ($\text{input}_i, \text{type}_i$)，其中 input_i 是 o_i 输入张量索引的列表， type_i 是运算符类型。若 μ 图的运算符按秩递增排列，则称其处于标准形式。Mirage 通过要求运算符按秩递增顺序添加（第 9 行和第 22 行）来只生成标准形式的 μ 图。这种方法不会剪枝掉任何有效解，因为每个 μ 图都可以通过重排运算符的顺序转换为标准形式。

此外，Mirage 利用抽象表达式技术来剪枝不满足特定约束的前缀，这将在 §4.3 中介绍。

4.2 线程图构建

尽管类似的嵌套生成策略也可以应用于线程图，但 Mirage 选择使用基于变换的方法来构建线程图（见 Figure 5 第三个面板以及 Algorithm 1 中的第 4–5 行），以缩减搜索空间。

Mirage 在构建线程图时应用运算符融合，通过尽可能在寄存器文件中重用张量来减少对共享内存的访问。例如，Mirage 将 Figure 3b 中的三个逐元素运算符 (Mul、

Table 1: Mirage 支持的运算符。第二列显示支持该运算符的图层次 (K、B 和 T 分别表示核图、块图和线程图)。最后一列定义每个运算符输出的抽象表达式, 其中 E 将张量映射到其抽象表达式。

μ 图 运算符	图层次	输出张量的抽象表达式
InIter	B	$E(\text{InIter}(X)) = E(X)$
OutSaver	B	$E(\text{OutSaver}(X)) = E(X)$
Matmul	K, B, T	$E(\text{Matmul}(X, Y)) = \text{sum}(k, \text{mul}(E(X), E(Y)))^1$
Sum	K, B, T	$E(\text{Sum}(d_r, k_r, X)) = \text{sum}(k_r, E(X))^2$
EwAdd	K, B, T	$E(\text{EwAdd}(X, Y)) = \text{add}(E(X), E(Y))$
EwMul	K, B, T	$E(\text{EwMul}(X, Y)) = \text{mul}(E(X), E(Y))$
EwDiv	K, B, T	$E(\text{EwDiv}(X, Y)) = \text{div}(E(X), E(Y))$
EwExp	K, B, T	$E(\text{EwExp}(X)) = \text{exp}(E(X))$
Repeat	K, B	$E(\text{Repeat}(X)) = E(X)$
Reshape	K, B	$E(\text{Reshape}(X)) = E(X)$
Sqr	K, B	$E(\text{Sqr}(X)) = \text{mul}(E(X), E(X))$
Sqrt	K, B	$E(\text{Sqrt}(X)) = \text{sqrt}(E(X))$
SiLU	K, B	$E(\text{SiLU}(X)) = \text{silu}(E(X))$
Accum	B	$E(\text{Accum}(X, m, i)) = \text{sum}(i, E(X))$ (若 $m = \phi$, 否则 $E(X)$) ³

¹ k 表示 A 最后一维 (即规约维度) 的大小。Matmul 在最内两个维度上执行, 前导维度进行批处理。

² 沿维度 d_r 对每 k_r 个元素求和。

³ 沿 fmap m 累积 i 次 for 循环迭代的结果。

Sqrt 和 Div) 融合到一个线程图中, 避免将中间结果保存到共享内存, 并将这些运算符的整个计算保持在寄存器文件中。虽然我们目前的实现侧重于运算符融合, 但也可以使用额外的基于规则的变换来构建线程图。

4.3 通过抽象表达式剪枝

在搜索可能的 μ 图空间时, 我们的目标是避免其中间结果无法对目标计算作出贡献的 μ 图前缀。例如, 对于输入程序 $X \cdot Z + Y \cdot Z$, 我们可以剪枝掉计算 $X \cdot Y$ 的前缀, 但不应剪枝计算 $X + Y$ 的前缀, 因为 $(X + Y) \cdot Z$ 与输入程序等价。然而, 在搜索某个计算的同时, 我们如何确定一个前缀是否能对该计算作出贡献呢? 下面, 我们通过抽象化来绕开这个“先有鸡还是先有蛋”的问题, 开发出一种由这一直觉驱动的剪枝技术。我们首先介绍抽象化方法——抽象表达式——然后解释它如何用于剪枝。最后, 我们在特定条件下提供该剪枝不会排除最优 μ 图的理论保证。

抽象表达式。 回顾一下, μ 图 中的一条边对应于输入张量的一个张量值函数。直观地, 抽象表达式通过忽略同一输入张量的元素之间的差异来对这些函数进行抽象。形式上, 抽象表达式是整数理论和未解释函数理论的一阶逻辑项。在 μ 图 中, 每条边的抽象表达式 (记为 $E(\cdot)$) 在 Table 1 中定义。在计算 μ 图 的抽象表达式时, 所有图定义运算符都被“内联”处理。具体来说, 为图定义运算符输入计算的表达式被传入其底层图, 该底层图的输出表达式即成为图定义运算符的输出表达式。Figure 6 展示了注意力计算的一个子图中的抽象表达式。

抽象子表达式与剪枝。 我们使用抽象表达式, 通过形式化两种关系——等价关系和抽象子表达式关系——来剪枝 μ 图 的搜索空间。具体地, 我们剪枝掉任何抽象表达式不是与输入程序的某个等价抽象表达式的子表达式的 μ 图 前缀。

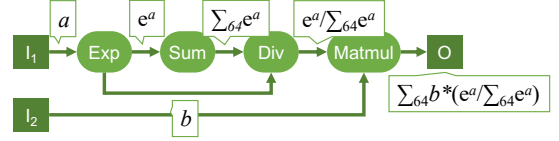


Figure 6: 抽象表达式示意图。张量的抽象表达式标注在边上。这里使用便于理解的记法: e^a 表示 $\exp(a)$, $\sum_k a$ 表示 $\text{sum}(k, a)$, a/b 表示 $\text{div}(a, b)$, $a * b$ 表示 $\text{mul}(a, b)$ 。张量 I_1 、 I_2 和 O 均为 64×64 矩阵。

我们将抽象表达式形式化为整数算术理论和未解释函数理论上的一阶逻辑中的未解释函数, 并使用 SMT 求解器基于 Table 2 中的两组公理 A_{eq} 和 A_{sub} 对其进行推理。

第一, A_{eq} 公理化了抽象表达式之间的等价关系。需要注意的是, 这些公理不必是可靠的——并不要求具有等价抽象表达式的 μ 图 在功能上也等价, 因为不等价的 μ 图 可能具有相同的抽象表达式。第二, A_{sub} 公理化了抽象表达式之间的子表达式关系。 A_{sub} 的一个关键性质是: 对于任何 μ 图 G_1 是 G_2 的前缀——即 G_2 可以通过用额外的运算符扩展 G_1 来构建——有 $E(G_1)$ 是 $E(G_2)$ 的抽象子表达式; 形式上, $A_{\text{sub}} \models \text{subexpr}(E(G_1), E(G_2))$, 其中 $\models$ 表示在整数算术和未解释函数理论下的蕴含。

在搜索过程中, Algorithm 1 首先计算输入 LAX 程序的抽象表达式 (记为 E_O), 并在 $A_{\text{eq}} \cup A_{\text{sub}} \models \text{subexpr}(E(G), E_O)$ 时剪枝任何 μ 图 前缀 G 。即, 若一个图的抽象表达式不是 E_O 的子表达式, 则将其剪枝。这一检查通过 SMT 求解器 (Z3 [18]) 执行。作为优化, 这些检查的结果会被缓存以供复用, 因为 Mirage 在搜索过程中可能遇到具有相同抽象表达式的多个 μ 图。

理论保证与剪枝-最优性权衡。 直观地, 我们的剪枝方法会保留所有可能引出其抽象表达式 (根据 A_{eq}) 与输入 LAX 程序等价的 μ 图 的前缀。形式上:

定理 1 (通过抽象表达式剪枝). 对于输入 μ 图 G_0 , 以及 G_0 等价的 μ 图 G , 若 $A_{\text{eq}} \models E(G_0) = E(G)$, 则 G 将被 Algorithm 1 生成。

Proof. 由 Tables 1 and 2, 我们证明对于任意运算符 op , 若 $Y = op(X_1, \dots, X_n)$, 则对 $1 \leq i \leq n$ 有 $A_{\text{sub}} \models \text{subexpr}(E(X_i), E(Y))$ 。即, op 每个输入的抽象表达式始终是 op 输出的子表达式。考虑到 A_{sub} 包含自反性和传递性公理, 对于任何 G' 是 G 的前缀, 有 $A_{\text{sub}} \models \text{subexpr}(E(G'), E(G))$ 。结合 $A_{\text{eq}} \models E(G_0) = E(G)$ 的假设, 得到 $A_{\text{eq}} \cup A_{\text{sub}} \models \text{subexpr}(E(G'), E(G_0))$ 。因此, G 的所有前缀都不会被剪枝, Mirage 将生成 G 。□

5 概率等价验证器

Mirage 的概率等价验证器用于检验候选 μ 图 是否与目标 LAX 程序等价。其核心思想是在两个有限域中对两者使用随机输入进行求值。使用有限域而非浮点数, 不仅避免了浮点误差, 还提供了强有力的理论保证: 接受不等价 μ 图 的概率可以被降至任意低。

Table 2: 用于剪枝的抽象表达式公理化。Mirage 通过查询 SMT 求解器检验 $\text{subexpr}(E_1, E_2)$ 是否被这些公理蕴含，来判断抽象表达式 E_1 是否为 E_2 的子表达式。所有公理中的变量均全称量化。

抽象表达式性质	说明
等价公理 A_{eq}	
$\forall x, y. \text{add}(x, y) = \text{add}(y, x)$	交换律
$\forall x, y. \text{mul}(x, y) = \text{mul}(y, x)$	交换律
$\forall x, y, z. \text{add}(x, \text{add}(y, z)) = \text{add}(\text{add}(x, y), z)$	结合律
$\forall x, y, z. \text{mul}(x, \text{mul}(y, z)) = \text{mul}(\text{mul}(x, y), z)$	结合律
$\forall x, y, z. \text{add}(\text{mul}(x, z), \text{mul}(y, z)) = \text{mul}(\text{add}(x, y), z)$	分配律
$\forall x, y, z. \text{add}(\text{div}(x, z), \text{div}(y, z)) = \text{div}(\text{add}(x, y), z)$	结合律
$\forall x, y, z. \text{mul}(x, \text{div}(y, z)) = \text{div}(\text{mul}(x, y), z)$	结合律
$\forall x, y, z. \text{div}(\text{div}(x, y), z) = \text{div}(x, \text{mul}(y, z))$	结合律
$\forall x. x = \text{sum}(1, x)$	恒等规约
$\forall x, i, j. \text{sum}(i, \text{sum}(j, x)) = \text{sum}(i * j, x)$	结合律
$\forall x, y, i. \text{sum}(i, \text{add}(x, y)) = \text{add}(\text{sum}(i, x), \text{sum}(i, y))$	结合律
$\forall x, y, i. \text{sum}(i, \text{mul}(x, y)) = \text{mul}(\text{sum}(i, x), y)$	分配律
$\forall x, y, i. \text{sum}(i, \text{div}(x, y)) = \text{div}(\text{sum}(i, x), y)$	分配律
$\forall x, y. \text{mul}(\text{exp}(x), \text{exp}(y)) = \text{exp}(\text{add}(x, y))$	分配律
$\forall x, y. \text{mul}(\text{sqrt}(x), \text{sqrt}(y)) = \text{sqrt}(\text{mul}(x, y))$	分配律
子表达式公理 A_{sub}	
$\forall x, y. \text{subexpr}(x, \text{add}(x, y))$	
$\forall x, y. \text{subexpr}(x, \text{mul}(x, y))$	
$\forall x, y. \text{subexpr}(x, \text{div}(x, y))$	
$\forall x, y. \text{subexpr}(y, \text{div}(x, y))$	
$\forall x. \text{subexpr}(x, \text{exp}(x))$	
$\forall x. \text{subexpr}(x, \text{sqrt}(x))$	
$\forall x. \text{subexpr}(x, \text{silu}(x))$	
$\forall x, i. \text{subexpr}(x, \text{sum}(i, x))$	
$\forall x. \text{subexpr}(x, x)$	自反性
$\forall x, y, z. \text{subexpr}(x, y) \wedge \text{subexpr}(y, z) \rightarrow \text{subexpr}(x, z)$	传递性

对于一般程序，随机测试几乎无法提供任何正确性保证。然而，我们证明了对于 LAX 程序（以下正式定义），随机测试提供了概率正确性保证，且重复测试可以将错误概率降低到任意小的阈值。

先前的工作 [46] 将类似的技术应用于只含线性运算符（如矩阵乘法、卷积）的张量程序的等价性检验。我们开发了一种也支持除法和指数运算的随机测试技术，这对许多 DNN 优化是必要的（例如 §3 中的 RMSNorm 示例）。

Mirage 对以下定义的 LAX μ 图（linear——线性，division——除法，an exponential——一个指数运算）验证等价性。我们在 §5.1 中介绍主要理论结果，并在 §5.2 中介绍 Mirage 的验证方法。

定义 5.1 (LAX μ 图). 若 μ 图 G 满足以下条件，则称其为 LAX μ 图：(1) G 只包含多线性运算符⁴、除法和指数运算，且 (2) G 中从输入到输出的每条路径上最多包含一次指数运算。

⁴具有 n 个输入的运算符 op 是多线性的，若 op 对所有输入 I_k 均为线性：(1) $\forall X, Y. op(I_1, \dots, I_{k-1}, X, I_{k+1}, \dots, I_n) + op(I_1, \dots, I_{k-1}, Y, I_{k+1}, \dots, I_n) = op(I_1, \dots, I_{k-1}, X + Y, I_{k+1}, \dots, I_n)$ ，以及 (2) $\alpha \cdot op(I_1, \dots, I_{k-1}, X, I_{k+1}, \dots, I_n) = op(I_1, \dots, I_{k-1}, \alpha \cdot X, I_{k+1}, \dots, I_n)$ 。

Table 3: 随机测试的算术运算。Mirage 选取两个满足 q 整除 $p - 1$ 的素数 p 和 q 。 x_p 和 x_q 分别是有限域 $\mathbb{Z}_p$ 和 $\mathbb{Z}_q$ 中的值。记号 x^{-1} 和 $\sqrt{x}$ 分别表示对应有限域中 x 的乘法逆元和平方根。具体地， $xx^{-1} \bmod p = 1$ 且 $\sqrt{x}\sqrt{x} \bmod p = x$ 。

运算	操作数 1	操作数 2	输出
加法	(x_p, x_q)	(y_p, y_q)	$((x_p + y_p) \bmod p, (x_q + y_q) \bmod q)$
减法	(x_p, x_q)	(y_p, y_q)	$((x_p - y_p) \bmod p, (x_q - y_q) \bmod q)$
乘法	(x_p, x_q)	(y_p, y_q)	$(x_p y_p \bmod p, x_q y_q \bmod q)$
除法	(x_p, x_q)	(y_p, y_q)	$(x_p y_p^{-1} \bmod p, x_q y_q^{-1} \bmod q)$
指数	(x_p, x_q)	—	$(\omega^{x_p} \bmod p, -)$
平方根	(x_p, x_q)	—	$(\sqrt{x_p}, \sqrt{x_q})$

5.1 理论基础

不失一般性，假设 LAX μ 图 G 接受 n 个输入张量并产生一个输出张量。我们的理论结果可以直接推广到具有多个输出的 LAX μ 图。由于每个 LAX μ 图包含线性运算符、除法以及每条路径上至多一次指数运算，输出张量每个元素的计算可以表达为以下形式（通过使用标准恒等式，如 $\frac{a}{b} = \frac{ad}{bc}$ ， $\frac{a}{b} + \frac{c}{d} = \frac{ad+bc}{bd}$ ， $e^x e^y = e^{x+y}$ ）：

$$\frac{\sum_{i=1}^k f_i \exp(g_i/h_i)}{\sum_{i=1}^{k'} f'_i \exp(g'_i/h'_i)} \quad (2)$$

其中 $f_i, g_i, h_i, f'_j, g'_j$ 和 h'_j ($1 \leq i \leq k, 1 \leq j \leq k'$) 是输入张量元素上的多项式。

支撑我们随机等价验证的主要理论结果是以下定理，它将有限域上的多项式恒等测试 (PIT) [37, 54] 推广到了 LAX μ 图。注意，两个 LAX μ 图之差也具有 Equation (2) 的形式。因此，两个 LAX μ 图的恒等测试归结为测试该形式的表达式是否为零。由于存在指数运算，我们使用两个有限域而非一个。⁵

定理 2. 设 P 是 Equation (2) 所描述形式的函数，其中 $f_i, g_i, h_i, f'_i, g'_i, h'_i$ 是整数系数在 $[-w, w]$ 范围内、次数至多为 d 的非零多项式。设 p, q 为素数，满足 $q|p-1$ 且 $q > 2w$ 。设 $\mathcal{G}$ 为 $\mathbb{Z}_p$ 中的 q 次单位根集合。若 P 不是零函数，则 [27]

$$\Pr_{(\vec{u}, \vec{v}, \omega) \leftarrow \mathbb{Z}_p^N \times \mathbb{Z}_q^N \times \mathcal{G}} \left[\frac{\sum_{i=1}^k f_i(\vec{u}) \omega^{g_i(\vec{v})/h_i(\vec{v})}}{\sum_{i=1}^{k'} f'_i(\vec{u}) \omega^{g'_i(\vec{v})/h'_i(\vec{v})}} \right] \leq 8dk^4/q + q^{-1/k^2}.$$

5.2 有限域上的随机测试

Mirage 利用 Theorem 2，通过在 Theorem 2 定义的有限域 $\mathbb{Z}_p$ 和 $\mathbb{Z}_q$ 上执行随机测试，来概率性地验证两个 μ 图的等价性。为了检验两个 μ 图的等价性，Mirage 首先生成输入张量，其中每个元素均匀采样自 $\mathbb{Z}_p \times \mathbb{Z}_q$ 。Mirage 还从 $\mathbb{Z}_p$ 中的 q 次单位根集合中均匀采样 ω ，用于指数运算。Mirage 随后使用 Table 3 中定义的运算对

⁵我们使用两个素数 p 和 q 分别对指数外和指数内的值进行多项式恒等测试 [37, 54]。条件“ q 整除 $p-1$ ”是为了确保 q 次单位根在 $\mathbb{Z}_p$ 中存在。

这些输入求值两个 μ 图。如 §5.1 中所述, $\mathbb{Z}_p$ 和 $\mathbb{Z}_q$ 分别用于指数外和指数内的计算。除指数运算外, 所有运算均通过在 $\mathbb{Z}_p$ 和 $\mathbb{Z}_q$ 中独立进行模运算来实现。对于指数运算, Mirage 使用 $\mathbb{Z}_q$ 中的值 x_q , 并计算 $\omega^{x_q} \bmod p$ 以得到 $\mathbb{Z}_p$ 中的结果。

注意, 在 LAX μ 图中, 每条路径上最多只进行一次指数运算。最后, Mirage 检查两个 μ 图是否产生相同的输出。该过程重复多次, 若两个 μ 图通过所有随机测试, 则认为它们等价。以下定理由 Theorem 2 推导, 表明该过程可以达到任意低的错误率。

定理 3. 等价的 μ 图始终通过 μ 图验证。对于两个不等价的 μ 图以及给定的概率阈值 $0 < \delta \leq 1$, 这两个 μ 图通过所有 $\Omega(\frac{k^2}{\ln q} \cdot \ln \frac{1}{\delta})$ 次随机测试的概率至多为 δ 。

数值稳定性。 虽然定理将有限域与实数计算联系起来, 但在浮点运算中——尤其是涉及大中间值导致的溢出或下溢时——可能出现与实数计算的差异。Mirage 使用浮点测试来过滤掉具有显著数值误差的 μ 图。

6 μ 图 优化器

对于每个经过验证的 μ 图, Mirage 的 μ 图优化器通过进一步执行布局优化、运算符调度和内存规划来最大化其性能, 如 Figure 1 所示。Mirage 将这些 μ 图优化推迟到验证之后, 原因有以下两点。第一, 这些优化不影响生成的 μ 图的正确性; 在生成 μ 图时省略它们, 可以缩小 Mirage 需要考察的搜索空间, 因为具有相同图拓扑但张量布局、运算符顺序或内存分配方案不同的 μ 图在 μ 图生成器看来是相同的。第二, 在验证之后再应用这些优化, 也缩小了这些优化的搜索空间, 因为 μ 图优化器只需优化与输入在功能上等价的 μ 图。

张量布局。 μ 图优化器在核、块和线程层次上探索所有中间张量可能的数据布局, 并选择最佳组合以最大化性能。我们将布局选择表述为一个约束优化问题, 并使用整数线性规划 (ILP) 算法进行最优求解。具体地, 对于每个张量 t 和 t 的每种可能布局 l , 引入一个布尔变量 $B_{t,l}$ 来表示张量 t 是否使用布局 l 。核、块和线程层次的运算符可能对张量布局施加各种约束。例如, 要使用 cuBLAS 库 [16] 的核进行矩阵乘法, 两个输入张量的最内维度必须在最后两个维度中。这些限制被转换为关于 $B_{t,l}$ 的一系列线性约束。不同的张量布局可能导致不同的性能。例如, 某些输入张量布局支持从设备内存到共享内存的批量拷贝, 而其他布局不支持。Mirage 引入一个代价函数来建模不同布局选择下每个运算符的性能。Mirage 使用现成的 ILP 求解器 (即 Z3 [18]) 来找到满足所有布局约束同时最小化代价的最优布局策略。

运算符调度。 在 μ 图中, 有多种拓扑顺序可以执行运算符, 不同的顺序可能产生不同的性能。对于给定的输入 μ 图, μ 图优化器通过最小化每个线程块内的线程级同步 (即 CUDA 中的 `__syncthreads()`) 来识别

Table 4: 评估中使用的 DNN 基准测试。

名称	描述	基础架构
GQA	分组查询注意力	LLaMA-3-70B [41]
QKNorm	带注意力的 QK 归一化	Chameleon-7B [40]
RMSNorm	带线性层的 RMS 归一化	LLaMA-2-7B [44]
LoRA	低秩适配	GPT-3-7B-LoRA [6]
GatedMLP	门控多层感知器	Falcon-7B [10]
nTrans	归一化 Transformer	nGPT-1B [28]

高效的运算符调度策略。为实现这一目标, Mirage 为每个节点标注一个深度, 定义为从任意输入运算符到该节点的最长路径长度。Mirage 使用动态规划算法计算每个节点的深度, 并按深度升序调度所有运算符。这种方法最小化了生成的 CUDA 核所需的线程级同步次数, 因为 Mirage 只需在深度不同的运算符之间插入同步点。

内存规划。 第三类验证后优化是内存规划, 用于确定核、块和线程层次所有中间张量的内存偏移量。Mirage 将内存规划表述为一个动态存储分配问题, 并穷举所有可能的分配方案以发现最优策略。

7 实现

Mirage 使用 C++、CUDA 和 Python 实现, 共约 30K 行代码。核运算符使用 cuDNN 和 cuBLAS 库 [15, 16] 实现, 块和线程运算符使用 cuTLASS [2] 和 CUDA PTX 实现。对于每个输入张量程序, Mirage 自动生成并验证潜在的 μ 图。对于每个经过验证的 μ 图, Mirage 为该 μ 图的所有自定义核生成 CUDA 源代码, 并使用 CUDA 编译器将代码编译为二进制文件。这种方法支持对一般张量程序的即时 (JIT) 编译和部署, 生成的核可以通过几行代码修改直接集成到 PyTorch 程序中。Mirage 的 SMT 和 ILP 求解器使用 Z3 4.12.6 [18] 实现。

我们的实现支持 Table 1 中列出的运算符。Mirage 可以扩展以包含新的运算符, 例如卷积或矩阵乘法的变体, 在核、块和/或线程层次上均可添加。为了支持一个新的线性运算符, Mirage 需要: (1) 该运算符在核、块和/或线程层次上的浮点实现, 用于 μ 图优化器生成 CUDA 核; (2) 该运算符的模运算实现 (见 §5); 以及 (3) 为该运算符扩展抽象表达式公理 A_{eq} 和 A_{sub} (见 §4.3)。

为了利用 Theorems 2 and 3, 随机测试应使用足够大的素数 p 和 q 并多次迭代。我们目前的实现使用乘积能放入 16 位整数的最大 p 和 q 值 (即 $p=227, q=113$) 在 GPU 上运行这些随机测试。我们利用 Mirage 的 GPU 优化 (如将中间结果保存在共享内存中) 来加速搜索过程。我们还执行一次不迭代的随机测试, 比较输出张量的所有元素。需要注意的是, 这一等价验证过程不会引入假阴性。虽然理论上可能引入假阳性, 但在实践中我们从未观察到这种情况。因此, 我们认为这一过程对于搜索过程已经足够, 并计划在优化过程最后只对最佳 μ 图添加一个提供理论保证的最终验证步骤。

非 Lax 程序的等价验证。 虽然 Mirage 可以为任意张量程序生成 μ 图, 但概率等价验证器仅限于 LAX 程序,

不支持某些 DNN 运算符（如 ReLU [32]）。作为替代方案，我们开发了一种基于求解器的验证器，用于处理任意张量程序。该验证器依赖于用户提供的各个运算符的数学性质（例如，线性性、结合律、交换律和分配律），这些性质以一阶逻辑形式定义，并使用自动定理证明器来验证等价性。与概率等价验证器相比，基于求解器的验证器支持更通用的程序，但需要为每个新运算符额外指定其性质。对基于求解器的验证器的详细讨论超出了本文的范围。

8 评估

8.1 实验设置

由于 Mirage 是针对 LAX 程序的超级优化器，我们的评估重点关注现有 DNN 中常用的各种 DNN 基准测试，每个基准测试都是一个 LAX 程序。这些基准测试提供了最细粒度的方式来比较 Mirage 和现有系统的性能。Table 4 展示了我们评估中使用的六个基准测试。GQA、RMSNorm 和 GatedMLP 是大型语言模型 (LLM) 的主要构建模块。QKNorm 在注意力之前引入查询-键归一化，以增强模型收敛性 [40]。LoRA 支持对 DNN 进行低秩适配以在不同任务上进行微调。对于 GQA，我们使用 8K 的上下文长度；对于 QKNorm，使用 4K 的上下文长度，分别对应 LLaMA-3-70B [41] 和 Chameleon-7B [40] 所支持的最大上下文长度。此外，我们还评估了 Mirage 生成的核对完整 DNN 端到端性能的提升，包括 Chameleon [40]、nGPT [28]、LLaMA-3 [41] 和 LoRA [22]。

实验在 NVIDIA A100 和 H100 GPU 上进行，每个 GPU 配有 40GB 内存。我们的所有基准测试都能在单个 GPU 上运行，但 GQA（用于 LLaMA-2-70B）通常需要使用张量模型并行 [39] 跨四个 GPU 并行化。因此，我们在此并行化策略下评估 GQA，其中八个键值头均等分配到四个 GPU 上。由于 Mirage 和所有基线的性能只取决于输入张量的形状，我们使用随机输入重复每个实验 1,000 次并报告平均运行时间。

我们的基准测试之一 LoRA 需要拼接操作来表达一种常见优化：通过拼接融合两个矩阵乘法。为了在 Mirage 中支持这种优化，我们引入了一个新的线性运算符，接受四个输入并计算 $f(W, X, Y, Z) = (W \parallel X) \times (Y \parallel Z)$ ，其中 $\parallel$ 表示张量拼接。该运算符等价于计算 $W \times Y + X \times Z$ 。我们为该运算符定义抽象表达式如下： $E(f(W, X, Y, Z)) = \text{add}(\text{sum}(k_1, \text{mul}(E(W), E(Y))), \text{sum}(k_2, \text{mul}(E(X), E(Z))))$ ，其中 k_1 和 k_2 分别是 W 和 X 的最后一维。

除非另有说明，Mirage 在核图中最多考虑 5 个运算符，在每个块图中最多考虑 11 个运算符。

8.2 基准测试结果

Figure 7 比较了 Mirage 与现有系统在 NVIDIA A100 和 H100 GPU 上六个 DNN 基准测试上的性能。所有系统均使用半精度浮点数运行这些 DNN 基准测试。TASO [25] 和 PET [46] 是自动生成核层次代数变换的

DNN 超级优化器。我们报告 TASO/PET 组合基线，因为最新的 TASO 实现将 PET 的部分等价变换包含为特殊替换规则。PyTorch [34] 使用高度优化的 cuDNN 和 cuBLAS 库 [15, 16] 在 GPU 上执行 DNN 运算符。对于 PyTorch 基线，我们启用 `torch.compile` 并使用 FlashAttention 核以最大化性能。TensorRT 及其 LLM 变体 TensorRT-LLM 包含一组为常见张量运算符（如注意力 [42]）手动设计和高度优化的核。FlashAttention 及其推理变体 FlashDecoding 是用于高效注意力的手写核 [17, 21]。最后，Triton 是一种基于调度的优化器，用于生成高性能核，已被生产系统采用，性能超过其他基于调度的方法 [43]。所有基线均使用 CUDA Graphs 以最小化核启动开销。

与最佳现有方法相比，Mirage 通过结合代数变换、调度变换和生成新的自定义核，将这些基准测试的性能提升了最高达 3.3 $\times$ 。§3 展示了 Mirage 为 RMSNorm 发现的最优 μ 图。以下我们对其余基准测试进行案例研究。

GQA。 分组查询注意力是 LLM 的核心组件，已被现有框架深度优化。例如，FlashAttention 和 FlashDecoding 是专家设计的注意力核，已被现有 LLM 推理系统采用 [17]。Mirage 发现了这些专家设计的核以及其他在某些场景下超越它们最高达 2.2 $\times$ 的 μ 图。加速通过以下两项在现有手写核基础上的额外优化实现。第一，当前方法依赖固定启发式策略来确定 GQA 的网格维度，在某些场景下并不最优。例如，TensorRT-LLM 在批大小为 1 和 8 时，分别以网格维度 (8, 2, 1) 和 (8, 2, 8) 启动 GQA 核。然而，这两种配置都无法充分利用 A100 (108 个 SM) 和 H100 (132 个 SM) GPU 上的所有 SM。相比之下，Mirage 自动为每个 μ 图搜索最佳网格维度，从而实现全部 SM 利用率。进一步的消融实验表明，当使用与 TensorRT-LLM 相同的网格维度时，Mirage 发现的最优 μ 图的性能降低了 18%。

第二，现有方法使用固定的张量维度来跨线程块并行化 GQA。例如，FlashAttention [17] 沿样本、头和查询序列维度并行化注意力，而 FlashDecoding 和 TensorRT-LLM 则利用样本、头和键值序列维度。这两种策略对于具有多头的传统多头注意力很高效，但对于头数较少的 GQA 则不够最优。相比之下，Mirage 通过在样本、KV 头、查询序列和键值序列维度之间进行选择，自动选择最高效的并行化策略。此外，Mirage 为不同的注意力场景生成不同的 μ 图，与现有系统使用的启发式策略相比，最多可减少 7 $\times$ 的设备内存访问。

在现有系统中实现 Mirage 的 μ 图是可行的，但需要大量工程工作来支持针对不同场景的不同核。相比之下，Mirage 会自动生成它们并验证其正确性。

QKNorm。 为减少模型发散，近期一些 DNN 在 Transformer 架构中引入了查询-键归一化 (QKNorm) [40]。QKNorm 在注意力之前对查询和键向量应用层归一化，如 Figure 8a 所示。这些额外的归一化层尚未被现有注意力实现（如 FlashAttention 和 TensorRT-LLM）支持，需要为归一化和注意力分别启动独立的核。

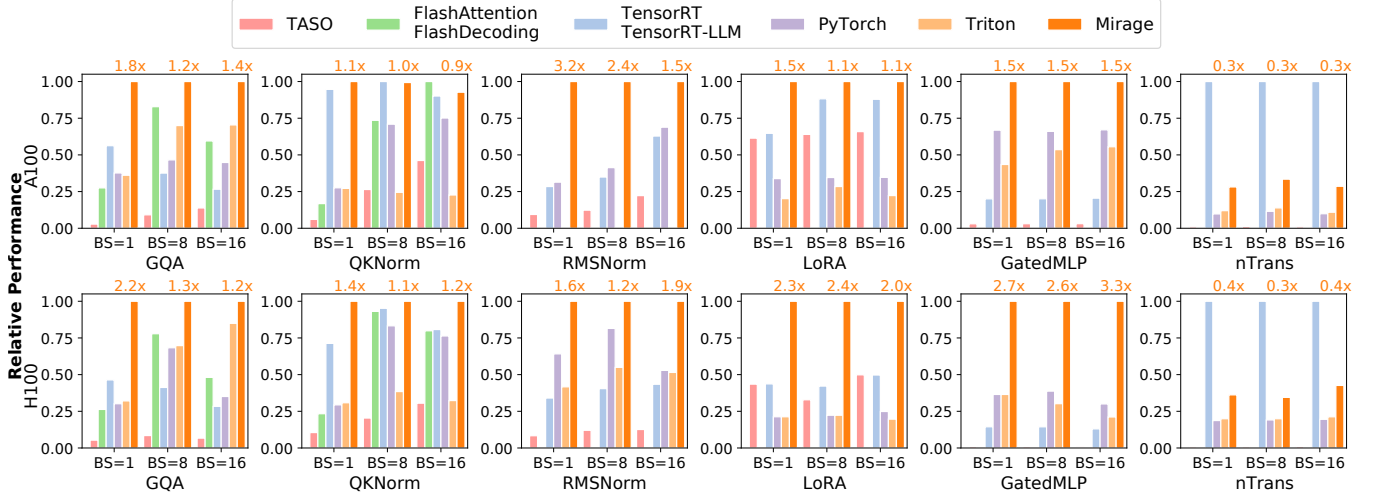
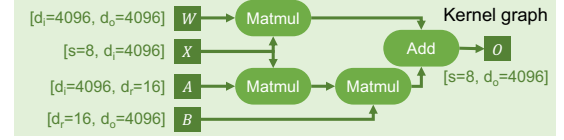
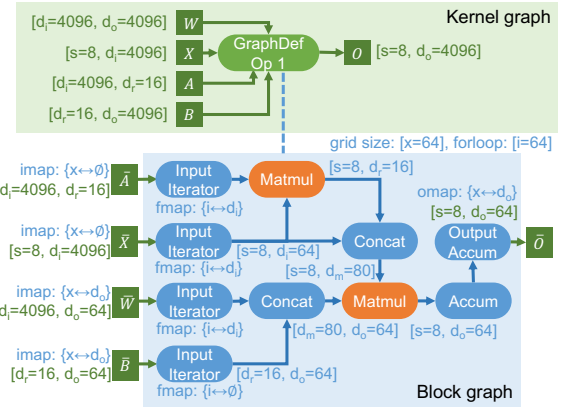


Figure 7: 在 A100 和 H100 GPU 上对 6 个基准测试进行 Mirage 与现有系统的比较。所有系统的性能均以 Mirage 为基准归一化（越高越好）。Mirage 柱状图上方的数字显示相对最佳基线的加速比。



(a) 现有系统中 LoRA 的核图。



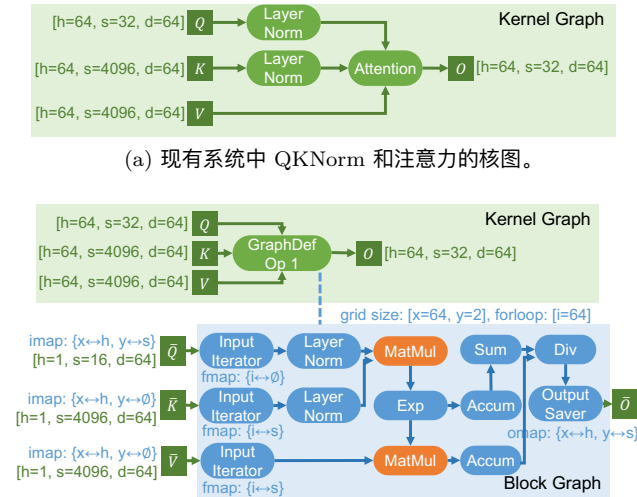
(b) Mirage 为 LoRA 发现的最优 μ 图。

Figure 9: 比较现有优化器和 Mirage 针对 LoRA 使用的张量程序: $O = W \times X + B \times A \times X$ 。注意矩阵 A 和 B 均为低秩矩阵。

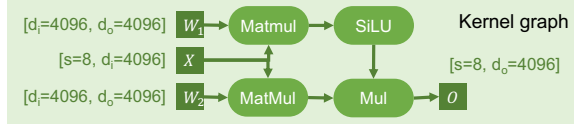
Mirage 自动发现了一个将 QKNorm 和注意力计算集成到自定义核中的 μ 图，如 Figure 8b 所示。该 μ 图重新组织了注意力计算以支持与两个层归一化的融合，避免将中间结果写入 GPU 设备内存，将核执行时间减少了高达 1.4x。

LoRA。 低秩适配 (LoRA) 向预训练 DNN 的线性运算符引入一对低秩适配器，以提升其在下游任务上的性能。现有张量程序优化器为原始线性运算符和 LoRA 引入的

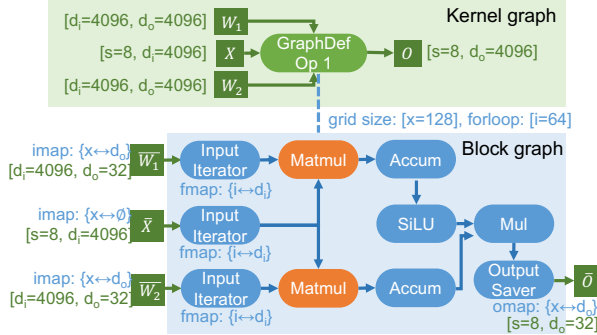
Figure 8: 比较现有优化器和 Mirage 针对 QKNorm 和注意力使用的 μ 图。



(b) Mirage 为 QKNorm 和注意力发现的最优 μ 图。



(a) GatedMLP 的核图。



(b) Mirage 为 GatedMLP 发现的最优 μ 图。

Figure 10: 比较现有优化器和 Mirage 针对 GatedMLP 使用的 μ 图。

两个额外线性运算符分别启动独立的核(Figure 9a),由于这些 LoRA 运算符涉及极少量计算,这会引入较高的核启动开销。Figure 9b 展示了 Mirage 为 LoRA 发现的最优 μ 图,它将三个 Matmul 和随后的 Add 融合到单个核中。Mirage 利用以下代数变换将计算重新组织为两个块级 Matmul: $W \times X + B \times A \times X = (W \| B) \times (X \| (A \times X))$ 。Figure 9b 中的 Concat 不涉及任何计算,通过更新 GPU 共享内存中的张量偏移量来实现。该 μ 图将 LoRA 的执行代价降低了 1.1–2.4 $\times$ 。

GatedMLP。 门控多层感知器广泛用于 DNN 中以捕获非线性表示。我们使用 Falcon-7B [10] 中引入的 GatedMLP 配置,其核图如 Figure 10a 所示。现有张量程序优化器通常将两个 Matmul 融合在单个核中以减少 GPU 设备内存访问,因为输入张量 X 只需加载一次。然而,这种方法仍需启动多个核并将中间结果——具体来说是两个 Matmul 的输出——存储在设备内存中,因为 SiLU 激活函数和逐元素乘法没有与 Matmul 融合。

相比之下, Mirage 发现的最优 μ 图 (Figure 10b) 在同一块图中并行执行两个 Matmul,并将其余计算(即 SiLU 和 Mul)作为同一块图内的后处理步骤进行融合。这种方法在 A100 GPU 上实现了 1.5 $\times$ 的加速,在 H100 GPU 上实现了 2.7–3.3 $\times$ 的加速。

nTrans。 为了加速模型训练, nGPT 引入了归一化 Transformer, 它对 Transformer 中的所有中间结果进行归一化 [28]。形式上, 计算定义为 $y = \text{Norm}(x + \alpha(\text{Norm}(h - x)))$, 其中 Norm 是归一化层, x , h 和 α 是输入张量。由于 nTrans 将归一化、逐元素加法和乘法交错进行, 现有系统为 nTrans 启动三个独立的核。Mirage 自动发现了一个将计算融合到单个核中并将所有中间结果存储在 GPU 共享内存中的 μ 图。Mirage 优于其他

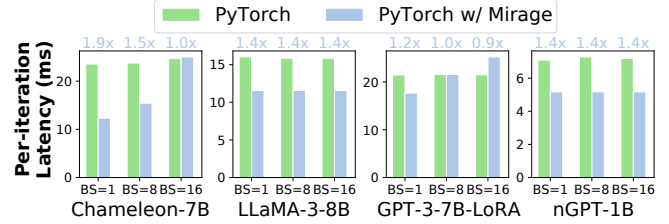


Figure 11: 比较 PyTorch 和使用 Mirage 生成核的 PyTorch 的端到端推理性能。

Table 5: Mirage 加速 μ 图生成技术的消融实验。我们评估多线程和抽象表达式对 RMSNorm 搜索时间的影响。

块图中算子最大数量	Mirage	Mirage (无多线程)	Mirage (无抽象表达式)
5	11 秒	58 秒	768 秒
6	16 秒	93 秒	19934 秒
7	22 秒	150 秒	> 10 小时
8	24 秒	152 秒	> 10 小时
9	26 秒	166 秒	> 10 小时
10	26 秒	166 秒	> 10 小时
11	28 秒	183 秒	> 10 小时

基线, 但比 TensorRT 慢。这一性能差距是因为 Mirage 在图定义核中为每个张量从全局内存加载数据到共享内存再写回。这种设计提高了内存效率并支持异步流水线, 但对于计算量轻的核, 这些内存传输的开销可能主导核的运行时间。为了缓解这一开销, 我们计划扩展 Mirage 以支持在数据加载时绕过共享内存, 从而避免不必要的数据移动。

8.3 端到端结果

除了微基准性能, 我们还评估了 Mirage 生成的核对于常用 DNN 端到端延迟的影响。Mirage 支持即时编译和部署, 其生成的核可以直接集成到 PyTorch 程序中。我们在四个 DNN 模型上比较了使用原生手写 CUDA 核的 PyTorch 和使用 Mirage 生成核的 PyTorch。Figure 11 展示了结果。Mirage 通过自动生成高度优化的核, 将这些模型的端到端延迟减少了 0.9–1.9 $\times$ 。这一改进只需对 PyTorch 程序进行几行代码修改即可实现。

8.4 搜索时间

在我们的评估中, Mirage 优化一个 LAX 程序最多需要 4 小时。这一优化是在目标硬件上部署之前的一次性代价。本小节提供 Mirage 搜索过程的详细结果和消融实验, 重点分析其技术如何在保持低搜索时间的同时支持对大型 μ 图的探索。具体地, 我们评估了两项技术的影响: 通过抽象表达式剪枝 (§4.3) 和多线程。Table 5 报告了 RMSNorm 在不同块图最大运算符数量下的搜索时间。

多线程显著缩短了搜索时间, 而通过抽象表达式剪枝对 Mirage 的可扩展性至关重要。具体地, 剪枝技术使

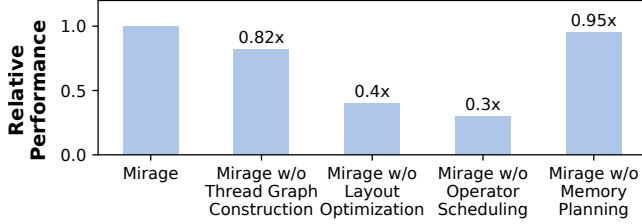


Figure 12: Mirage 中优化手段的消融实验。我们在批大小为 1 的 GQA 基准测试上，在 A100 GPU 上评估分别禁用每项优化时的性能下降。

Mirage 能够探索每个块图最多包含 11 个运算符的 μ 图，而禁用抽象表达式剪枝则将 Mirage 限制在 10 小时搜索窗口内只能处理最多 6 个运算符的块图。需要注意的是，发现 Figure 3 所示的 RMSNorm 优化 μ 图需要探索包含 11 个运算符的块图。

8.5 优化手段消融实验

我们进行消融实验，评估线程图构建和 §6 中介绍的优化手段（包括布局优化、运算符调度和内存规划）的影响。具体地，我们测量在独立禁用每项优化时 Mirage 发现的最优 μ 图的性能下降情况。实验在批大小为 1 的 GQA 基准测试上的 A100 上进行。结果如 Figure 12 所示，禁用任何单项优化都会导致 5% 到 70% 不等的性能下降。

9 相关工作

手动设计的核。 许多现有框架，如 TensorFlow XLA [1, 9]、PyTorch [34] 和 TensorRT [42]，依赖 GPU 专家为机器学习运算符手动设计核。近期，大量工程努力被投入到手动优化常用 DNN（尤其是基础模型 [12]）的 GPU 核中。例如，为加速注意力计算 [47]，基于 FlashAttention [4, 5, 17, 21] 开发了多种专用核。由于现代 GPU 日益复杂——例如 A100 中的张量核心 [29] 和 H100 中的线程块簇 [7]——手动设计的核可能会错过难以手动发现的微妙优化。

基于超级优化的方法。 超级优化最初被引入以寻找最优指令序列 [11, 30, 36]。近期工作将超级优化技术应用于张量程序 [23–26, 45, 46, 49, 52]。然而，所有这些工作都只考虑核层次的代数变换，无法发现需要在核、块和线程所有层次联合考虑代数变换和调度变换的更复杂优化。我们的评估表明，Mirage 大幅优于现有 DNN 超级优化器，证明了多层次联合优化的重要性。

基于调度的方法。 近期工作引入了能自动优化核 GPU 执行调度的机器学习编译器。TVM [13, 14]、Ansor [51] 和 Triton [43] 等系统及其他工作 [19, 20, 53]，均基于 Halide 中引入的算法-调度分离思想，搜索在 GPU 上执行用户指定算法的优化调度。然而，基于调度的方法要

求用户明确指定每个核的算法，其性能受限于所提供算法的质量。

多层次图表示。 Welder [38] 和 ASPEN [33] 引入了多层次块图，与 Mirage 的 μ 图有相似之处，因为两种表示都遵循 GPU 层次结构。然而，先前工作侧重于调度变换，而 Mirage 通过同时考虑代数变换和发现新的自定义核，超越了单纯的调度优化。本文中提出的大多数优化方案超出了这些先前方法的范畴。

10 结论

本文提出了 Mirage，这是第一个面向张量程序的多层次超级优化器。Mirage 引入了一种层次图表示方法，用于在 GPU 执行层次结构的核 (kernel)、线程块 (thread block) 和线程 (thread) 三个层次上描述张量程序，并使用一种基于抽象的新型剪枝技术来显著缩小 Mirage 需要考察的搜索空间，同时提供一定的最优性保证。即使对于被广泛使用和深度优化的深度神经网络，Mirage 也能以最高达 3.3 $\times$ 的倍数超越现有张量程序优化器的性能。

致谢

感谢匿名评审人和我们的指导人 Stephanie Wang 提供的宝贵意见和建议。感谢 Tianqi Chen、Phillip Gibbons、Bohan Hou、Muyan Hu、Jinchen Jiang、Xiaoyu Jiang、Ruihang Lai、Yu Zhou 以及其他 CMU Catalyst 成员对本研究的反馈。本研究部分受 NSF 项目 CNS-2147909、CNS-2211882 和 CNS-2239351 的资助，以及 Amazon、Cisco、Google、Meta、NVIDIA、Oracle、Qualcomm 和 Samsung 的研究资助支持。本研究还部分受魏茨曼科学研究院新科学家中心研究资助以及 Azrieli 基金会资助的支持。

References

- [1] Xla: Optimizing compiler for tensorflow. <https://www.tensorflow.org/xla>, 2017.
- [2] Nvidia/cutlass: Cuda templates for linear algebra subroutines. <https://github.com/NVIDIA/cutlass>, 2019.
- [3] Tensorflow graph optimization with grappler. https://www.tensorflow.org/guide/graph_optimization, 2019.
- [4] Transformer related optimizations. <https://github.com/NVIDIA/FasterTransformer>, 2020.
- [5] Flash-decoding for long-context inference. <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>, 2023.

- [6] Llama-7b-lora. <https://huggingface.co/Laurie/llama7b-lora-merged/tree/main>, 2023.
- [7] Nvidia h100 tensor core gpu. <https://www.nvidia.com/en-us/data-center/h100/>, 2023.
- [8] A Triton implementation of the FlashAttention2 algorithm. <https://triton-lang.org/main/getting-started/tutorials/06-fused-attention.html>, 2023.
- [9] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. Tensorflow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, OSDI, 2016.
- [10] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Merouane Debbah, Etienne Goffinet, Daniel Heslow, Julien Launay, Quentin Malartic, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. Falcon-40B: an open large language model with state-of-the-art performance. 2023.
- [11] Sorav Bansal and Alex Aiken. Automatic generation of peephole superoptimizers. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS XII, 2006.
- [12] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models, 2022.
- [13] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q. Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: end-to-end optimization stack for deep learning. *CoRR*, abs/1802.04799, 2018.
- [14] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems 31*, NeurIPS’18, 2018.
- [15] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cudnn: Efficient primitives for deep learning. *CoRR*, abs/1410.0759, 2014.
- [16] Dense Linear Algebra on GPUs. <https://developer.nvidia.com/cublas>, 2016.
- [17] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference, 2023.
- [18] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, TACAS’08/ETAPS’08, 2008.
- [19] Siyuan Feng, Bohan Hou, Hongyi Jin, Wuwei Lin, Junru Shao, Ruihang Lai, Zihao Ye, Lianmin Zheng, Cody Hao Yu, Yong Yu, and Tianqi Chen. Tensorir: An abstraction for automatic tensorized program optimization, 2022.

- [20] Bastian Hagedorn, Bin Fan, Hanfeng Chen, Cris Cecka, Michael Garland, and Vinod Grover. Graphene: An ir for optimized tensor computations on gpus. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS 2023, page 302–313, New York, NY, USA, 2023. Association for Computing Machinery.
- [21] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhang Dong, and Yu Wang. Flashdecoding++: Faster large language model inference on gpus, 2024.
- [22] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [23] Muyan Hu, Ashwin Venkatram, Shreyashri Biswas, Balamurugan Marimuthu, Bohan Hou, Gabriele Oliaro, Haojie Wang, Liyan Zheng, Xupeng Miao, Jidong Zhai, and Zhihao Jia. Optimal kernel orchestration for tensor programs with korch. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, page 755–769, New York, NY, USA, 2024. Association for Computing Machinery.
- [24] Byungsoo Jeon, Mengdi Wu, Shiyi Cao, Sunghyun Kim, Sunghyun Park, Neeraj Aggarwal, Colin Unger, Daiyaan Arfeen, Peiyuan Liao, Xupeng Miao, Mohammad Alizadeh, Gregory R. Ganger, Tianqi Chen, and Zhihao Jia. Graphpipe: Improving performance and scalability of dnn training with graph pipeline parallelism. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS '25, page 557–571, New York, NY, USA, 2025. Association for Computing Machinery.
- [25] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. Taso: Optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, page 47–62, New York, NY, USA, 2019. Association for Computing Machinery.
- [26] Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. In *Proceedings of the 2nd Conference on Systems and Machine Learning*, SysML'19, 2019.
- [27] Jiayu Li and Mengdi Wu. Identity testing for circuits with exponentiation gates, 2025.
- [28] Ilya Loshchilov, Cheng-Ping Hsieh, Simeng Sun, and Boris Ginsburg. nGPT: Normalized transformer with representation learning on the hypersphere, 2024.
- [29] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter. Nvidia tensor core programmability, performance & precision. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, May 2018.
- [30] Henry Massalin. Superoptimizer: a look at the smallest program. In *ACM SIGARCH Computer Architecture News*, volume 15, 1987.
- [31] Ravi Teja Mullapudi, Andrew Adams, Dillon Sharlet, Jonathan Ragan-Kelley, and Kayvon Fatahalian. Automatically scheduling halide image processing pipelines. *ACM Trans. Graph.*, 35(4), 2016.
- [32] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ICML'10, pages 807–814, USA, 2010. Omnipress.
- [33] Jongseok Park, Kyungmin Bin, Gibum Park, Sangtae Ha, and Kyunghan Lee. Aspen: Breaking operator barriers for efficient parallelization of deep neural networks. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 68625–68638. Curran Associates, Inc., 2023.
- [34] Tensors and Dynamic neural networks in Python with strong GPU acceleration. <https://pytorch.org>, 2017.
- [35] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '13, 2013.
- [36] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. In *ACM SIGPLAN Notices*, volume 48, 2013.

- [37] J. T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, oct 1980.
- [38] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. Welder: Scheduling deep learning memory access via tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 701–718, Boston, MA, July 2023. USENIX Association.
- [39] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-Lm: Training multi-billion parameter language models using model parallelism. *CoRR*, abs/1909.08053, 2019.
- [40] Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models, 2024.
- [41] The Llama 3 team. The llama 3 herd of models, 2024.
- [42] NVIDIA TensorRT: Programmable inference accelerator. <https://developer.nvidia.com/tensorrt>, 2017.
- [43] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, page 10–19, New York, NY, USA, 2019. Association for Computing Machinery.
- [44] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [45] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Patrick S. McCormick, Jamaludin Mohd-Yusof, Xi Luo, Dheevatsa Mudigere, Jongsoo Park, Misha Smelyanskiy, and Alex Aiken. Unity: Accelerating DNN training through joint optimization of algebraic transformations and parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, pages 267–284. USENIX Association, 2022.
- [46] Haojie Wang, Jidong Zhai, Mingyu Gao, Zixuan Ma, Shizhi Tang, Liyan Zheng, Yuanzhi Li, Kaiyuan Rong, Yuanyong Chen, and Zhihao Jia. PET: Optimizing tensor programs with partially equivalent transformations and automated corrections. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 37–54. USENIX Association, July 2021.
- [47] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art machine learning for pytorch, tensorflow, and jax. <https://github.com/huggingface/transformers>, 2022.
- [48] Yichen Yang, Phitchaya Mangpo Phothilimtha, Yisu Remy Wang, Max Willsey, Sudip Roy, and Jacques Pienaar. Equality saturation for tensor graph superoptimization, 2021.
- [49] Yichen Yang, Phitchaya Phothilimthana, Yisu Wang, Max Willsey, Sudip Roy, and Jacques Pienaar. Equality Saturation for Tensor Graph Superoptimization. *Proceedings of Machine Learning and Systems*, 3:255–268, March 2021.
- [50] Biao Zhang and Rico Sennrich. Root mean square layer normalization, 2019.
- [51] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor : Generating high-performance tensor programs for deep learning. *CoRR*, abs/2006.06762, 2020.
- [52] Liyan Zheng, Haojie Wang, Jidong Zhai, Muyan Hu, Zixuan Ma, Tuowei Wang, Shuhong Huang, Xupeng Miao, Shizhi Tang, Kezhao Huang, and Zhihao Jia. EINNET: Optimizing tensor programs with Derivation-Based transformations. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 739–755, Boston, MA, July 2023. USENIX Association.
- [53] Size Zheng, Yun Liang, Shuo Wang, Renze Chen, and Kaiwen Sheng. Flextensor: An automatic schedule exploration and optimization framework for tensor computation on heterogeneous system. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’20, page 859–873, New York, NY, USA, 2020. Association for Computing Machinery.

- [54] Richard Zippel. Probabilistic algorithms for sparse polynomials. In *International symposium on symbolic and algebraic manipulation*, pages 216–226. Springer, 1979.